EvoLLM: High-Coverage, Readable, and Token-Efficient Unit Test Generation for Industrial Java Codebases

Seokhyeon Moon
Samsung SDS & Korea University
Seoul, Republic of Korea
shyeon.mun@samsung.com

Myungho Lee
Samsung SDS
Seoul, Republic of Korea
frango.lee@samsung.com

Jihun Kim
Samsung SDS
Seoul, Republic of Korea
jihun1.kim@samsung.com

Yoon-Chan Jhi
Samsung SDS
Seoul, Republic of Korea
yoonchan.jhi@samsung.com

Jihoon Cho
Samsung SDS
Seoul, Republic of Korea
jihoon1.cho@samsung.com

Hakjoo Oh✉
Korea University
Seoul, Republic of Korea
hakjoo_oh@korea.ac.kr

Abstract

Adopting automated Java unit-test generation in industrial settings requires high coverage, readability, token efficiency, and modern-Java deployability. No existing technique meets all four: search-based test generators struggle with readability and modern-Java deployability, LLM-centric generators consume tens of millions of tokens, and prior LLM-based test refactoring methods drop baseline coverage by 8–21 percentage points (pp) in our controlled setups.

We report on EvoLLM, a hybrid Java unit-test generator we built and deployed to address all four requirements. EvoLLM combines EvoSuite$_{ng}$—a failure-analysis-driven hardening of EvoSuite that resolves deployability issues and strengthens baseline coverage—with cluster-wise LLM-based refactoring that groups tests by coverage goals and rewrites each cluster in a single LLM call grounded in compact program-under-test context. Across six open-source and three industrial projects, EvoLLM outperforms six state-of-the-art baselines, improving line coverage by 8.2–20.7 pp, branch coverage by 6.2–19.4 pp, and mutation score by 6.8–24.5 pp, while reducing LLM token usage by 47.3%–93.0%. In addition, the test generation stage of EvoLLM demonstrates robust deployability across target projects built with JDKs 8, 11, and 17, while achieving strong effectiveness on a Spring-core release requiring JDK 17. Finally, a 14-participant developer survey further suggests that EvoLLM-generated tests ranked higher in readability than those from prior tools and developer-written tests.

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

automated unit test generation, search-based software testing, large language models, test refactoring, token efficiency



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3834525

ACM Reference Format:
Seokhyeon Moon, Myungho Lee, Jihun Kim, Yoon-Chan Jhi, Jihoon Cho, and Hakjoo Oh. 2026. EvoLLM: High-Coverage, Readable, and Token-Efficient Unit Test Generation for Industrial Java Codebases. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3834525

1 Introduction

Unit tests are essential for preventing regressions and supporting safe software evolution, but maintaining comprehensive unit-test suites becomes increasingly difficult as software systems and development organizations scale [12, 60]. In our industrial setting, thousands of developers continuously merge changes into large codebases that support critical business services, where undetected regressions can propagate quickly and are costly to recover from. In this paper, we report on our experience deploying automated unit-test generation in this setting and present EvoLLM, a hybrid Java unit-test generator developed through this effort that combines search-based test generation with large language model (LLM)-based refactoring.

Design Goals. Our goal is to design an automated unit-test generator that satisfies the requirements for industrial adoption:

- ***High coverage***: Our internal continuous integration workflow enforces strict coverage requirements, such as 70% line coverage for each class. A generator falling below this bar turns automation into additional manual work rather than a net savings.
- ***Readability***: Tests have to be easy to maintain. We noticed that if developers struggle to understand the generated code, they simply delete and rewrite it from scratch, making the tool useless.
- ***Token efficiency***: At our scale, generation must use LLM tokens predictably and sparingly. Because input and output token counts directly affect API cost and inference throughput [5, 37], our deployment environment imposes per-user token budgets and throughput limits.

Beyond these core requirements, real-world adoption also depends on ***deployability on modern Java projects***. Our internal services are built on modern stacks, including JDK 17, Spring-based frameworks, and Gradle-based build configurations. For our teams, a test-generation tool is not deployable unless it can generate, compile, and execute tests under such project settings. Although this

requirement is primarily an engineering hurdle, satisfying it was a strict prerequisite for deployment in our setting.

Existing Techniques. To determine if existing techniques could meet these criteria, we evaluated three distinct families of test-generation techniques. Each showed theoretical promise but ultimately failed under our constraints.

First, *search-based software testing (SBST)* tools failed to meet our readability and deployability requirements. For example, EvoSuite [18]—a de facto SBST tool for Java—targets coverage within a fixed time budget, but its generated tests are often difficult for developers to understand [11, 53]. In our setting, outputs with opaque identifiers (`test0`, `var1`), dense assertion chains, and few comments often led developers to simply rewrite the tests by hand. Deployability proved to be an equally critical blocker: standard EvoSuite failed to produce compilable tests under JDK 17 and exhibited systemic integration failures on modern framework code such as Spring. Compounding these limitations, even its primary strength—coverage—was insufficient on real industrial code, yielding only 22% branch coverage on an internal 50 KLOC project.

Second, *LLM-centric generators* [4, 9, 36, 41, 49, 50, 59, 61] that directly prompt an LLM to synthesize tests from the target program can produce readable tests, but fail on both efficiency and reliability. By repeatedly resending overlapping context, they consumed tens of millions of tokens in our trials, making them economically unviable at scale. Furthermore, they frequently hallucinated unsupported behaviors, resulting in 20–60% execution-failure rates on non-trivial classes— effectively breaking both our token efficiency and coverage requirements.

Third, *LLM-based refactoring methods* [13, 58] that use an LLM to refactor generated tests can improve readability, but they still suffer from token-efficiency issues and can reduce test coverage substantially. This stemmed from three structural limitations: (1) they refactor tests in isolation without direct program-under-test context, forcing the model to infer intent from opaque literals and yielding misleading assertions; (2) their per-test, multi-stage prompting consumes nearly as many tokens as direct generation, destroying the expected cost advantage; and (3) the LLM-based refactoring phase reduced the original SBST coverage by 8–21 percentage points (pp) in our controlled setups.

Our Approach. To overcome these limitations, we present EvoLLM, a hybrid Java unit-test generator designed to simultaneously satisfy our industrial requirements. EvoLLM achieves this by combining a hardened EvoSuite, namely EvoSuite$_{ng}$, with a context-efficient LLM-based refactoring. EvoSuite$_{ng}$ directly resolves the deployability and baseline coverage failures of EvoSuite through targeted engineering improvements derived from extensive failure analysis. To improve readability while keeping LLM-token use low and predictable, our refactoring stage groups generated tests into behavior-focused clusters—tests whose coverage goals lie mainly in the same program-under-test (PUT) method—and rewrites each cluster in a single LLM call grounded in compact PUT context. This design helps avoid misleading rewrites and substantially reduces coverage degradation and LLM-token use while producing readable tests.

We evaluate EvoLLM on six open-source projects from the SBST competition benchmark [22, 45]—Fastjson, Guava, Seata, Spoon, Okio, and Weka—and three internal industrial projects. We compare EvoLLM against an SBST tool (EvoSuite), three LLM-centric generators (ChatUniTest [9], CoverUp [4], HITS [56]), and two LLM-based refactoring methods built atop EvoSuite (UTGen [13], CLAST [58]). Across the nine subjects, EvoLLM improves average line coverage by 8.2–20.7 pp, branch coverage by 6.2–19.4 pp, and mutation kill ratio by 6.8–24.5 pp over the compared techniques, while using only 13.0M total LLM tokens, reducing token usage by 47.3%–93.0% relative to the other LLM-based approaches. In separate deployability analyses, EvoSuite$_{ng}$ remains stable across JDK 8–17 on Seata and Spoon, with execution-failure rates of at most 0.38%, and achieves 71.1% line and 65.0% branch coverage on Spring-core 6.0.18, a major framework requiring JDK 17. A 14-participant user study further suggests that respondents rated EvoLLM-generated tests favorably in terms of readability and contextual understandability.

Table 1: Motivating observations from our preliminary experience with EvoSuite in our setting.

Program(s)	Setting	Observation	Implication
Open-source (Seata 0.5.0, Spoon 7.0.0)	JDK 8	52.9% average branch coverage	Insufficient for practical adoption in our internal workflow
	JDK 11	43.9% average branch coverage	Effectiveness already degrades in a newer Java environment
	JDK 17	0.0% average branch coverage; 0% of classes with a compilable test	Analysis fails entirely, although JDK 17 support is essential in our setting
Industrial project (50 KLOC)	JDK 8	22% average branch coverage; generated tests were difficult to understand and extend	Developers tended to rerun EvoSuite after changes rather than review and evolve the generated tests

Contributions. In summary, we contribute:

- Four concrete requirements — high coverage, readability, token efficiency, and modern-environment deployability — distilled from our industrial deployment experience, against which we show that no existing off-the-shelf technique succeeds.
- EvoLLM, a hybrid unit-test generator whose central idea is cluster-wise LLM refactoring of an SBST-generated suite, grounded in compact PUT context and executed in a single call per cluster.
- EvoSuite$_{ng}$, a failure-analysis-driven, engineering-hardened variant of EvoSuite that improves test effectiveness and supports modern-Java deployability across projects using JDKs 8, 11, and 17 including Spring Framework, which requires JDK 17.
- An extensive evaluation on six open-source and three industrial projects, comparing EvoLLM against SBST, LLM-centric, and LLM-based refactoring baselines along effectiveness, efficiency, readability, and deployability.

2 Motivation

We describe the practical limits of adopting SBST directly (Section 2.1) and the need for behavior-focused inputs with compact program context in post-generation refactoring (Section 2.2).

2.1 Coverage, Deployability, and Readability Challenges for SBST in Industrial Use

In our setting, software projects are evaluated through a centralized testing platform, where release readiness is assessed against

organization-wide quality requirements such as code coverage. Accordingly, a practical unit-test generation technique had to satisfy three requirements at once: sufficient coverage, deployability on modern Java projects, and readable tests that developers could inspect, debug, and extend.

Table 1 summarizes the main observations from our preliminary assessment of EvoSuite, a widely used SBST tool [9, 18, 20, 56]. For cross-version compatibility analysis, we used the SBST'22 benchmark subjects that remained compilable under JDK 8, 11, and 17; among the benchmark projects we considered, only Seata and Spoon satisfied this condition, yielding 30 classes in total. On these subjects, EvoSuite achieved 52.9% average branch coverage under JDK 8 and 43.9% under JDK 11, while it did not produce compilable tests under JDK 17. These results indicate that, even on benchmark subjects compatible with newer JDKs, EvoSuite did not provide the coverage and deployability required in our setting. Because a substantial portion of our internal projects rely on recent Spring Framework versions, for which JDK 17 support is essential, this compatibility gap was a practical barrier to adoption.

Even when generated tests were executable, their value as developer-facing artifacts remained limited because improving coverage required developers to understand and extend them. When an internal development team applied EvoSuite to their financial-service project of approximately 50 KLOC, EvoSuite achieved only 22% average branch coverage, far below the level expected in our internal workflow. Raising coverage toward the required quality level would have required them to extend the generated tests to cover the remaining behaviors. In follow-up discussions, however, they explained that this effort was difficult because the tests were hard to read and extend. Generated method and variable names rarely conveyed the test intent, and behavior-relevant statements were often entangled with incidental ones, making test failures or exceptions hard to diagnose. As a result, the generated suites were rarely used as maintainable artifacts; instead, developers mainly used them as disposable regression checks, rerunning EvoSuite after program changes rather than reviewing and evolving the tests.

These observations suggested that improving industrial applicability required more than simply reusing an existing SBST tool as-is. Since the limitations spanned coverage, deployability on modern Java projects, and developer-facing readability, our design required targeted improvements in both test generation and test readability.

2.2 The Need for Effective and Efficient Inputs for LLM-Based Test Refactoring

Post-generation LLM refactoring [13, 58] can improve readability while invoking the model more selectively than an LLM-centric generator. The effectiveness and efficiency of this approach, however, depend critically on two design choices in the refactoring prompt: which generated tests to include, and what PUT context to attach alongside them. In this section, we use the example in Figures 1 and 2 to derive three requirements that the refactoring prompt must satisfy. Together, these requirements motivate the cluster-wise refactoring design in Section 3.

Figure 1 shows a simplified `AccountStore` class whose `withdraw` method takes an account identifier and an amount. The method

```
1  class AccountStore {
2    static final int HIGH = 50;
3    private final Map<Long, Integer> bal = new HashMap<>();
4    private final Set<Long> verified = new HashSet<>();
5    private int audit, small;
6
7    void open(long accId, int b) { bal.put(accId, b); }
8    void verifyOtp(long accId, int code) {
9      if (code == 0xFE) verified.add(accId);
10   }
11   private boolean validateOtp(long accId) {
12     if (!verified.contains(accId)) return false; // g5T,g5F
13     audit++;                                     // g6
14     return true;
15   }
16   boolean withdraw(long accId, int amount) {
17     if (amount <= 0) throw new IllegalArgumentException(); // g1
18     int b = bal.get(accId);
19     if (b < amount) return false;            // g2
20     if (amount > HIGH) {                     // g4T,g4F
21       if (!validateOtp(accId)) return false;
22     } else { small++; }                      // g7
23     bal.put(accId, b - amount);              // g3
24     return true;
25   }
26 }
```

Figure 1: Program under test used in the motivating example.

```
1  @Test public void test0() {
2    AccountStore s = new AccountStore();
3    s.open(2024050612L, 100);
4    try { s.withdraw(2024050612L, 0); } catch (Throwable t) {} // g1
5    assertFalse(s.withdraw(2024050612L, 200));                 // g2
6    s.verifyOtp(2024050612L, 0xFE);
7    assertTrue(s.withdraw(2024050612L, 60)); // g4T,g5F,g6,g3
8  }
9
10 @Test public void test1() {
11   AccountStore s = new AccountStore();
12   s.open(2023050601L, 100);
13   assertTrue(s.withdraw(2023050601L, 10)); // g4F,g7,g3
14 }
15
16 @Test public void test2() {
17   AccountStore s = new AccountStore();
18   s.open(2026050611L, 100);
19   s.verifyOtp(2026050611L, 0xFE);
20   assertTrue(s.withdraw(2026050611L, 60)); // g4T,g5F,g6,g3
21 }
```

Figure 2: Illustrative EvoSuite tests generated from the PUT.

contains exceptional behavior, insufficient-balance behavior, a low-value withdrawal path, and a high-value withdrawal path guarded by one-time password (OTP) validation. We refer to the relevant structural goals as follows: g_1 (exception when `amount <= 0`), g_2 (return `false` when balance is insufficient), g_3 (update the balance), g_{4T}/g_{4F} (branch outcomes of `amount > HIGH`), g_{5T}/g_{5F} (branch outcomes in `validateOtp`), g_6 (increment the audit counter), and g_7 (increment the small-withdrawal counter).

Requirement 1. PUT context is necessary because the test body alone under-specifies the scenario. In Figure 2, calls such

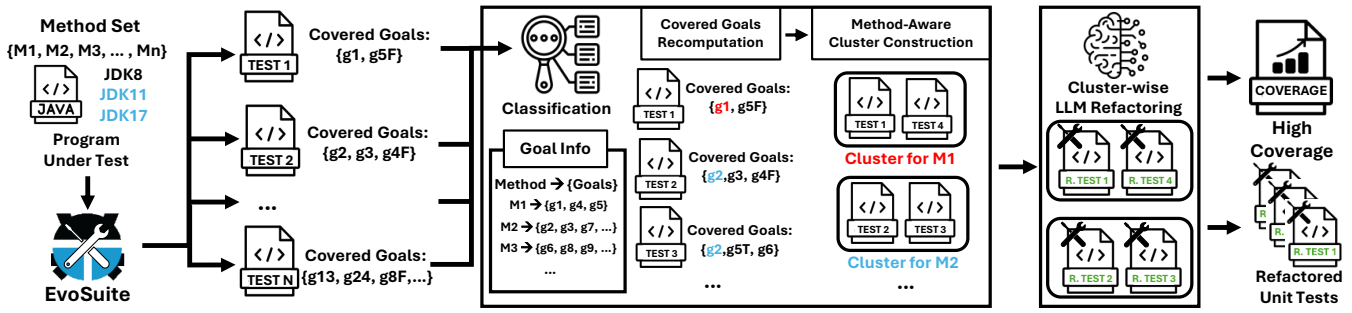


Figure 3: Overview of our approach.

as `s.withdraw(2024050612L, 60)` contain large numeric literals that may be read as date-like values, opaque transaction identifiers, or account identifiers. Only the PUT in Figure 1 reveals that the first argument is an account identifier used to retrieve the target account before applying the withdrawal. The ambiguity becomes more severe when a generated test mixes multiple behaviors. For example, `test0` combines the invalid-amount case (g_1), the insufficient-balance case (g_2), and the verified high-value withdrawal path (g_{4T}, g_{5F}, g_6, and g_3) in one test body, so the test alone does not reveal which behavior a refactoring should foreground. Without the PUT, the model must infer both the roles of the values and the primary scenario of the test from the body alone, making accurate refactoring substantially harder. The refactoring prompt must therefore include PUT context alongside the test.

Requirement 2. PUT context must be compact because attaching the full PUT to every test is inefficient and can be counterproductive. A naive way to satisfy Requirement 1 is to provide the entire PUT with each generated test, but doing so inflates prompt size and dilutes the context most relevant to the behavior being refactored. Different tests may cover the same structural goal while representing different primary behaviors, and therefore require different PUT regions. For example, both `test1` and `test2` cover the balance-update goal g_3, but they reach it through different scenarios. `test1` reaches g_3 through the low-value path (g_{4F}, g_7, g_3), whereas `test2` reaches it through the high-value verified path (g_{4T}, g_{5F}, g_6, g_3), which additionally traverses OTP-validation logic. Thus, `test1` is better treated as a low-value withdrawal scenario, whereas `test2` is better treated as an OTP-validation-related scenario; their refactoring inputs should therefore expose different PUT regions rather than the same full PUT.

Requirement 3. Compact context should be shared across related tests because extracting it per test is itself inefficient. Tests that exercise related behaviors of the same primary method typically need overlapping PUT regions: for example, high-value withdrawal tests need the high-value branch and the OTP logic, even if their finer-grained scenarios differ. Treating them independently re-extracts and re-sends the same PUT regions, but naively reusing one shared context across unrelated tests would re-introduce the dilution problem from Requirement 2. The refactoring prompt should therefore be constructed at the cluster level, sharing overlapping PUT context across related tests while still identifying each test's primary behavior.

These three requirements jointly shape our approach. In Section 3.2, Requirements 1 and 2 are addressed by identifying the behavior each test should foreground and extracting compact PUT context relevant to that behavior. Requirement 3 is addressed through method-aware clustering, which groups related tests so that compact PUT context can be shared across them.

3 Approach

Figure 3 summarizes EvoLLM. Given a PUT, EvoLLM proceeds in three stages. First, EvoSuite$_{ng}$ generates an initial test suite through a hardened SBST stage for coverage-oriented generation and deployability on modern Java projects. Second, EvoLLM converts the generated suite into behavior-focused clusters and extracts compact PUT context for each cluster. Third, EvoLLM refactors each cluster with a single LLM request.

3.1 Hardened Search-Based Test Generation

EvoLLM first uses EvoSuite$_{ng}$, an engineering-hardened EvoSuite-based generator that incorporates and extends the coverage- and deployability-oriented improvements developed in our prior work [35], to generate an initial test suite for the subsequent LLM-based refactoring pipeline. This hardening addresses two practical limitations of EvoSuite identified in Section 2.1: generation failures that reduced coverage and compatibility failures that prevented deployment on modern Java projects.

Generation Hardening for Improved Coverage. To identify generation failures and derive actionable engineering fixes, we ran EvoSuite on 70 classes from the SBST'20 Java unit-testing benchmark [15, 42], repeating each class ten times. Among the resulting 700 runs, 297 achieved branch coverage below 50%. We used these runs as a diagnostic pool, inspecting generated tests, compilation and execution outcomes, and the generation steps responsible for the observed failures. We then applied fixes incrementally and reran the benchmark, treating any branch-coverage drop exceeding 10 pp as a regression requiring inspection.

This analysis showed that low coverage was not merely due to hard-to-cover branches or insufficient search effort. Instead, many failures followed three recurring patterns: missed compatible inputs, unsound or overly conservative type reasoning, and repeated exploration of recursive object-instantiation paths that had already failed under the relevant recursion budget. EvoSuite$_{ng}$ addresses these failures through stricter type reasoning, more robust object

and variable construction, and pruning of redundant recursive-instantiation attempts.

Generic-heavy subjects such as Guava and Spoon exposed the importance of this hardening. For example, in `com.google.common.graph.Graphs`, non-compilable tests arose from incomplete reasoning about generic methods whose bounds depend on owner-class type variables, such as `<U extends T>` in `Owner<T extends Number>`. The original EvoSuite implementation instantiated `U` from a return-derived substitution map and checked its bound without the full owner-class type context. EvoSuite$_{ng}$ instead validates method type parameters in the owner-class type context, reducing both missed compatible inputs and non-compilable generic instantiations. EvoSuite$_{ng}$ also strengthens object construction by improving compatibility-aware input generation and pruning redundant recursive-instantiation attempts that have already failed under the current recursion budget.

Compatibility Hardening for Modern Java Projects. Beyond generation quality, industrial adoption required EvoSuite to be deployable on modern Java projects, including services using recent JDKs, Spring-based frameworks, and Gradle-based build configurations. Many target services rely on Spring Framework 6.x, which requires Java 17+ [54]. We therefore reviewed JDK 8–17 language, class-file, runtime, and module-system changes to identify required compatibility updates. For source-level and class-file features such as local-variable type inference, switch expressions, text blocks, records, sealed classes, and pattern matching [39], class-level test generation did not require redesigning EvoSuite's opcode-level analyses: these features either compile to existing JVM instructions or appear as class-file metadata handled by an updated bytecode parsing layer [40]. Accordingly, EvoSuite$_{ng}$ updates bytecode handling for JDKs 11 and 17 class files while preserving EvoSuite's core generation logic.

JPMS required separate handling because it affects module visibility, class loading, and reflection [46], especially under JDK 17's stronger encapsulation of JDK internals [47]. Because our goal is class-level white-box test generation rather than module-boundary conformance, the test harness opens and exports module packages before loading target classes, allowing EvoSuite$_{ng}$ to analyze and exercise non-exported classes without target-specific manual module configuration. In addition, for Gradle-based builds, EvoSuite$_{ng}$ derives the source layout and classpath needed by the generation–compilation–execution pipeline. Deployment on recent Spring-based projects also exposed additional compatibility failures involving restricted reflective or unsafe operations under newer JDKs [47], residual module-visibility compilation errors, and code-generation errors involving mocked classes. Overall, these fixes make EvoSuite$_{ng}$ substantially more robust on modern Java projects.

3.2 Test Cluster Construction

This stage converts the test suite generated by EvoSuite$_{ng}$ into compact inputs for cluster-wise LLM refactoring. For each goal covered by the generated suite, EvoLLM selects and minimizes a goal-focused test, i.e., a test that exposes the goal with relatively little additional behavior. The final goal-to-test mapping defines each test's assigned goals—the goals the test should foreground during refactoring—which are used to determine the test's primary target method.

Algorithm 1 Goal-Focused Test Cluster Construction

Require: Generated suite S, goals G covered by S, program under test P
Ensure: Clusters C, compact contexts $\mathcal{X}$
1: $W \leftarrow S$ ▷ working pool
2: $\mu \leftarrow \emptyset$ ▷ goal-to-test mapping
3: **for each** goal $g \in G$ **do**
4: $C_g \leftarrow \{ t \in W \mid g \in \text{Goals}(t) \}$
5: $t \leftarrow \textsc{SelectGoalFocusedTest}(C_g, g)$
6: $u \leftarrow \textsc{MinimizePreservingGoal}(t, g)$
7: $\text{Goals}(u) \leftarrow \textsc{RecomputeCoveredGoals}(u)$
8: $\mu(g) \leftarrow u$ ▷ g is mapped to the minimized test
9: $W \leftarrow W \cup \{u\}$
10: **end for**
11: $(S_{\text{ref}}, \text{Assigned}) \leftarrow \textsc{DeriveAssignedGoals}(G, \mu)$
12: **for each** test $t \in S_{\text{ref}}$ **do**
13: $\text{Primary}(t) \leftarrow \textsc{PrimaryMethod}(\text{Assigned}(t))$
14: **end for**
15: $C \leftarrow \emptyset$
16: **for each** method $m \in \{ \text{Primary}(t) \mid t \in S_{\text{ref}} \}$ **do**
17: $L_m \leftarrow \{ t \in S_{\text{ref}} \mid \text{Primary}(t) = m \}$
18: $C \leftarrow C \cup \textsc{SplitIfNeeded}(L_m)$
19: **end for**
20: **for each** cluster $c \in C$ **do**
21: $\mathcal{X}(c) \leftarrow \textsc{ExtractCompactContext}(P, c, \text{Primary}, \text{Goals})$
22: **end for**
23: **return** $(C, \mathcal{X})$

Goal-Focused Test Selection and Minimization. For each goal $g \in G$, Algorithm 1 identifies the tests in the current working pool W that cover g (Line 4). Here, $\text{Goals}(t)$ denotes the set of goals covered by test t, and $\text{Goals}_m(t) = \{ h \in \text{Goals}(t) \mid \text{Method}(h) = m \}$ denotes the subset of those goals that belong to method m. Among the tests in C_g, SelectGoalFocusedTest selects a goal-focused test for g by minimizing the lexicographic ranking $(|\text{Goals}(t)|, |\text{Goals}_{\text{Method}(g)}(t)|, |t|)$. The ranking prefers tests that cover fewer goals overall, fewer additional goals in the target method, and fewer statements when the first two criteria are tied. This discourages tests that reach g only as part of a larger mixed scenario and gives the LLM a clearer refactoring unit. Line 6 minimizes the selected test while preserving the target goal g. We reuse EvoSuite's test-case minimization primitive, which removes statements one at a time and keeps a deletion only if the resulting test still covers g. After minimization, EvoLLM recomputes the goals covered by the minimized test (Line 7), maps g to it (Line 8), and adds it to the working pool (Line 9).

In our implementation, we apply this goal-processing loop twice (Lines 3–10): the first pass creates minimized tests from the original suite, and the second pass revisits the expanded pool so that later goals can select already-simplified tests.

Deriving Assigned Goals. After goal-focused selection and minimization, DeriveAssignedGoals returns S_{ref}, the tests to be refactored, and the assigned-goal mapping Assigned (Line 11). The procedure first collects the candidate goal-focused tests $T = \{ \mu(g) \mid g \in G \}$ and filters out coverage-subsumed tests to reduce refactoring targets while preserving the goals covered by T. Specifically, if $\text{Goals}(t_i) \subset \text{Goals}(t_j)$ for $t_i, t_j \in T$, then t_i is filtered out and its mapped goals are reassigned to the subsuming test t_j. For each test $t \in S_{\text{ref}}$, $\text{Assigned}(t)$ denotes the goals assigned to t after this filtering step.

```
Refactor the following generated JUnit tests to improve readability. For each
test, infer the primary scenario, propose a descriptive test method name, and
rewrite the body using clearer names and explicit Given-When-Then comments.
Constraints: do not reduce achieved coverage; preserve compilability and the
behavior exercised by the original test.

[Program] <compact PUT context for the cluster> [/Program]
[Tests] <test_1> ... <test_k> [/Tests]

Return JSON only:
[ { "id": 1, "name": "withdrawSmallAmountUpdatesBalance", "test": "..." },
{ "id": 2, "name": "withdrawHighAmountWithVerifiedOtp", "test": "..." } ]
```

Figure 4: Simplified prompt for cluster-wise refactoring.

Method-Aware Clustering. To share PUT context across related tests, EvoLLM defines each test's primary target method as the PUT method containing the largest number of its assigned goals: $\text{Primary}(t) = \arg\max_m |\{ g \in \text{Assigned}(t) \mid \text{Method}(g) = m \}|$ (Line 13). The algorithm then forms method groups $L_m = \{ t \in S_{\text{ref}} \mid \text{Primary}(t) = m \}$ (Line 17). If L_m is within the configured cluster-size bound (4 in our implementation), SplitIfNeeded keeps it as a single cluster; otherwise, it partitions L_m into bounded-size clusters. This bound keeps each refactoring request small enough to reduce prompt complexity and keep the LLM focused on the behavior being refactored.

When splitting is needed, EvoLLM groups tests with similar method-local covered goals because such tests tend to require overlapping PUT regions. Let $n = |L_m|$, and let $df(x) = |\{ t \in L_m \mid x \in \text{Goals}_m(t) \}|$ be the number of tests in the method group whose method-local goal sets contain goal x. Each goal is assigned the smoothed inverse-document-frequency (IDF) weighting [28, 33] $w(x) = \log \frac{n+1}{df(x)+1} + 1$. For two method-local goal sets A and B, EvoLLM computes the weighted Jaccard similarity [26] $wJ(A, B) = (\sum_{x \in A \cap B} w(x)) / (\sum_{x \in A \cup B} w(x))$. This weighting emphasizes goals that distinguish behavior within the same method group, so tests exercising similar method-local behavior are more likely to remain in the same cluster.

Extracting Compact PUT Context. Finally, ExtractCompactContext constructs the PUT context used in each LLM refactoring request (Line 21). For a cluster c produced from method group L_m, EvoLLM collects the method-local goals covered by the clustered tests: $G_c^{ctx} = \bigcup_{t \in c} \text{Goals}_m(t)$. It retains the PUT regions (i.e., basic blocks) associated with G_c^{ctx}—a goal may correspond to more than one block—and elides unrelated regions with "...". Thus, the prompt contains the primary-method regions associated with the clustered tests rather than the full PUT, reducing prompt size and keeping the refactoring focused on cluster-relevant behavior. Algorithm 1 returns the resulting clusters and compact contexts for cluster-wise LLM refactoring.

3.3 Cluster-Wise LLM Refactoring

After cluster construction, EvoLLM submits one refactoring request per cluster, containing the assigned tests and their compact PUT context. Figure 4 shows the simplified prompt structure. The prompt instructs the model to infer the primary scenario exercised by each test, propose a descriptive test method name, and rewrite the test body using clearer names and explicit Given–When–Then comments [17]. It further requires the rewritten tests to remain compilable, preserve the behavior exercised by the original tests, and avoid coverage-reducing changes. The full prompt also provides a complete input–output example, omitted from Figure 4 for brevity, to clarify the expected rewrite and JSON format.

Upon receiving an LLM response, EvoLLM first validates the returned JSON structure and then compiles and executes each rewritten test. If parsing, compilation, or execution fails, EvoLLM retains the corresponding original test, thereby preserving the previously achieved coverage.

EvoLLM uses the LLM only for one-shot readability-oriented rewriting; it neither repairs failed rewrites nor generates additional tests. This design keeps LLM usage low and predictable, as each cluster incurs exactly one request and the total number of requests is bounded by the suite size and the maximum cluster size. We do not simply retry failed rewrites because resampling provides no additional program context. Meaningful recovery would instead require extracting more context and issuing a new request, adding cost and complexity that we consider unjustified for a readability-oriented transformation with a safe fallback. As discussed in Section 4, even under this non-iterative policy, EvoLLM incurs substantially less coverage degradation than prior techniques.

Table 2: Subject programs used in the evaluation.

Category	Subject	Project LOC	Target Classes	Target Methods
Open source	Spoon 7.0.0	120,285	15	278
	Guava 26	86,274	15	256
	Seata 0.5.0	25,082	15	196
	Fastjson 1.2.50	157,610	20	326
	Okio 1.16.0	17,524	7	222
	Weka 3.8.5	341,021	25	771
Industry	Client	10,500	30	186
	Server	35,164	30	131
	Common	8,564	30	268

4 Evaluation

In this section, we address the following research questions.

- **RQ1: Practical Effectiveness.** How effective is EvoLLM for repeated Java unit test generation in terms of deployability, test effectiveness, and generated-test stability?
- **RQ2: LLM Inference Efficiency.** How efficiently does EvoLLM use LLM tokens relative to prior LLM-based approaches?
- **RQ3: Ablation and Controlled Comparison.** What is the effect of EvoSuite$_{ng}$ and the proposed refactoring pipeline? How does EvoLLM compare with other refactoring approaches under a common EvoSuite baseline?
- **RQ4: Readability.** How do developers assess the tests produced by EvoLLM relative to tests produced by other approaches?

4.1 Experimental Setup

Our experimental setup is built around JUGE, an established infrastructure for benchmarking Java unit-test generators [14]. We chose JUGE because it has been used in multiple Java unit-testing tool competitions [15, 22, 27, 29, 45] and evaluates generators at the class level, matching our industrial setting where a centralized testing platform uses class-level coverage as a quality-gate signal.

Subjects. As shown in Table 2, our evaluation covers nine programs with 187 classes, comprising six open-source benchmark

subjects and three industrial projects. The open-source subjects were drawn from the two most recent SBST Java benchmark sets publicly available through JUGE at the time of our study, namely SBST'21 [45] and SBST'22 [22]. When a project appeared in both editions, we retained only the SBST'22 version to avoid duplication. These benchmark sets were originally curated to cover multiple application domains and exclude trivial or non-testable classes through complexity- and testability-based filtering before random sampling [22, 45]. For the industrial subjects, we randomly sampled 30 classes from each of three complementary projects: a cryptography-related client, a corresponding server, and a shared library providing common utility functions for their communication workflow.

Compared Techniques. We compare EvoLLM against state-of-the-art unit-test generation techniques spanning an SBST baseline (EvoSuite [18]), LLM-centric approaches (ChatUniTest [9], abbreviated as ChatUT in tables, HITS [56], and CoverUp [4]), and LLM-based refactoring approaches (UTGen [13] and CLAST [58]). Because no author-released implementation was available for HITS, and because CoverUp originally targets Python, we used the Java adaptations implemented in the ChatUniTest framework. In addition, since CLAST refines existing generated tests rather than generating tests from scratch, we applied it to EvoSuite-generated tests in our evaluation and measured the resulting end-to-end outcome. For the ablation study, we evaluate EvoSuite$_{ng}$ and our proposed refactoring pipeline built on top of EvoSuite.

Protocol, Budget, and Infrastructure. All techniques were evaluated under the same JUGE-based class-level protocol. Accordingly, we adapted method-oriented techniques—ChatUniTest, HITS, and CoverUp—to the same protocol: for each class, they generate tests for individual methods sequentially until the class-level time budget is exhausted. For LLM-based techniques, we disabled parallel LLM requests to avoid throughput differences under the same time budget. Except for these adjustments, we used each technique's default configuration. All techniques therefore followed the same end-to-end pipeline for generation, compilation, execution, and metric evaluation.

We used a 30-minute per-class wall-clock budget for all techniques, based on preliminary budget experiments on all 187 target classes. This budget was selected because the preliminary experiments showed that coverage gains had largely saturated by the 30-minute limit. For LLM-centric generators (i.e., ChatUniTest, HITS, and CoverUp), we used ChatUniTest as a representative configuration: its average branch coverage increased from 34.1% at 3 minutes to 45.1% at 18 minutes, but only to 45.9% at 30 minutes. For techniques that rely on SBST for initial test generation (i.e., UTGen, CLAST, and EvoLLM), increasing the EvoSuite search budget from 60 to 300 seconds changed average branch coverage only from 41.9% to 42.1%. We therefore applied the same 30-minute cap to every technique: LLM-centric generators used this cap for their end-to-end generation, repair, compilation, and validation workflow, whereas techniques relying on SBST used the default 60-second EvoSuite search budget and the remaining time for downstream phases such as post-processing, compilation, validation, and LLM-based refactoring when applicable.

Table 3: Deployability results of EvoSuite$_{ng}$.

Subject	Setting	Line coverage	Branch coverage	Exec. failure (%)
Seata 0.5.0	JDK 8	62.1	53.0	0.38
	JDK 11	61.9	53.3	0.00
	JDK 17	58.1	48.8	0.00
Spoon 7.0.0	JDK 8	85.4	79.7	0.06
	JDK 11	85.1	79.8	0.06
	JDK 17	82.8	78.6	0.00
Spring-core 6.0.18	JDK 8	–	–	–
	JDK 11	–	–	–
	JDK 17	71.1	65.0	2.01

We used gpt-oss-20b for all LLM-based techniques because its compact, open-weight design offers a practical balance between inference cost and generation quality for repeated unit-test generation in industrial settings [38]. Experiments were conducted on a machine equipped with a 64-core AMD Ryzen Threadripper PRO 5995WX CPU and 512 GB RAM, while LLM inference was served on a dedicated cloud machine equipped with two NVIDIA H100 GPUs. Each class-level experiment was repeated three times, resulting in 2,524.5 budgeted single-threaded execution hours in total (9 configurations × 187 classes × 3 repetitions × 0.5 hours).

Metrics. We report line coverage, branch coverage, mutation score—the percentage of mutants killed by the generated test suite—and execution-failure rate, defined as the proportion of generated test methods that fail during execution, e.g., due to uncaught runtime exceptions or failing assertions. For LLM-based techniques, we also report prompt tokens, completion tokens, total tokens, and total tokens per generated test method. The first three metrics quantify the absolute inference tokens incurred by LLM-based techniques across all model calls, while total tokens per generated test method normalizes these tokens by the amount of generated test code.

4.2 RQ1: Practical Effectiveness

Deployability. Table 3 reports deployability results for EvoSuite$_{ng}$, the generation stage of EvoLLM. We assess deployability at the EvoSuite$_{ng}$ stage because compatibility with newer JDKs is primarily determined by the test generation, compilation, and execution pipeline before LLM-based refactoring. On Seata and Spoon, which we use to assess cross-JDK behavior, EvoSuite$_{ng}$ produced stable results from JDK 8 to JDK 17 with very low execution-failure rates. Although coverage decreases slightly on JDK 17 for both subjects, these low execution-failure rates indicate that the generated tests and execution pipeline remain robust on projects compiled with newer JDKs. We also include a case study on Spring-core 6.0.18, which requires JDK 17, because many of our industrial projects rely on the Spring Framework. From 1,103 classes in Spring-core, 692 target classes remained after excluding inner classes and `package-info` entries. Across these classes, EvoSuite$_{ng}$ achieved 71.1% line coverage, 65.0% branch coverage, and a 2.0% execution-failure rate, suggesting that the engineering hardening extends beyond cross-JDK compatibility and remains applicable to large modern Java frameworks.

Effectiveness. Table 4 shows that EvoLLM exhibits the strongest effectiveness among the compared techniques. Across the nine subjects, it attains the best result on six subjects for line coverage, eight subjects for branch coverage, and six subjects for mutation

Table 4: Subject-wise line coverage, branch coverage, mutation score, and execution-failure rate.

Metric	Subject	EvoSuite	ChatUT	HITS	CoverUp	UTGen	CLAST	EvoLLM
Line cov.	Fastjson	63.6	56.5	61.2	47.0	54.5	55.1	**65.5**
	Guava	58.3	73.0	82.7	64.6	64.6	50.4	**84.9**
	Seata	**62.7**	60.8	61.0	57.6	44.3	54.5	61.1
	Spoon	54.5	71.6	56.8	55.7	59.4	49.6	**84.8**
	Okio	**79.1**	77.5	63.6	65.8	65.5	68.0	78.9
	Weka	**61.8**	50.5	40.6	27.4	49.5	52.8	59.0
	Client	55.9	46.4	43.3	31.0	50.4	47.5	**56.7**
	Server	52.6	66.1	55.3	55.1	51.2	40.8	**70.6**
	Common	81.6	70.6	72.1	56.7	78.3	64.5	**85.6**
	Average	63.3	63.7	59.6	51.2	57.5	53.7	**71.9**
Br. cov.	Fastjson	58.3	48.6	51.3	38.3	48.7	49.6	**59.4**
	Guava	51.0	70.5	82.5	63.2	62.2	42.3	**83.4**
	Seata	**57.4**	52.8	54.2	48.3	38.3	44.7	52.8
	Spoon	47.3	67.7	45.8	43.9	54.6	43.0	**79.5**
	Okio	74.2	75.5	62.0	62.2	59.7	57.5	**75.6**
	Weka	48.6	39.7	28.7	18.0	40.9	39.8	**49.4**
	Client	36.6	34.9	32.7	21.2	31.7	29.5	**39.0**
	Server	17.7	34.4	21.1	20.8	15.9	17.1	**35.6**
	Common	32.3	34.4	33.6	23.3	30.6	25.8	**38.7**
	Average	47.0	50.9	45.8	37.7	42.5	38.8	**57.1**
Mut. score	Fastjson	42.9	38.6	**45.3**	31.2	33.8	30.0	41.7
	Guava	44.4	50.5	59.5	46.5	47.1	28.6	**69.1**
	Seata	43.5	41.6	**45.1**	39.6	27.9	20.3	40.8
	Spoon	31.5	50.2	41.0	37.8	37.1	12.8	**61.4**
	Okio	59.2	**64.2**	53.8	52.5	42.8	35.2	54.0
	Weka	26.5	17.9	13.5	10.7	22.7	6.7	**27.0**
	Client	39.4	29.7	27.8	24.4	33.0	29.9	**40.1**
	Server	36.4	39.0	37.1	35.3	36.0	20.9	**45.2**
	Common	61.3	39.3	42.7	32.7	61.9	41.2	**67.3**
	Average	42.8	41.2	40.6	34.5	38.0	25.1	**49.6**
Exec. failure	Fastjson	0.90	33.90	29.20	24.20	5.20	**0.50**	4.20
	Guava	**0.00**	25.50	15.30	22.90	0.90	**0.00**	1.80
	Seata	4.60	35.70	33.70	29.90	15.40	35.50	**4.10**
	Spoon	4.20	58.00	33.10	30.40	19.10	34.10	**2.40**
	Okio	**0.00**	9.90	9.80	11.50	4.10	0.20	1.20
	Weka	**0.70**	27.40	31.10	26.50	4.10	18.00	1.90
	Client	**0.00**	30.00	30.80	14.80	1.60	0.10	1.50
	Server	**0.40**	32.60	30.30	15.80	2.00	0.80	2.40
	Common	**0.20**	53.30	33.20	14.20	1.20	0.30	0.90
	Average	**1.24**	34.03	27.39	21.12	5.96	9.95	2.27

score. This trend is observed in both the open-source benchmarks and the industrial subjects. In particular, EvoLLM achieves the best line, branch, and mutation results on Server and Common, and substantially outperforms all baselines on Spoon. These results indicate that EvoLLM improves not only structural effectiveness, as measured by line and branch coverage, but also fault-detection capability, as approximated by mutation score.

Stability. Among executable generated tests, EvoLLM keeps execution-failure rates low. Across all subjects, its rates are in the same low single-digit range as EvoSuite and lower on average than prior LLM-based baselines (Table 4). The category-level pattern is also consistent: LLM-centric baselines have the highest failure rates, while LLM-based refactoring baselines are lower but still above EvoLLM. This pattern likely reflects the role assigned to the LLM: LLM-centric approaches synthesize tests from scratch, which leaves room for hallucinated assertions, invalid inputs, unsupported call sequences, and inconsistent object states. LLM-based refactoring baselines start from SBST-generated tests and therefore fail less often, but model-driven refactoring can still introduce behavior-affecting edits. EvoLLM also starts from SBST-generated tests, but constrains rewriting with compact, scenario-specific PUT context and explicit validity requirements, helping preserve runtime consistency. This matters in industrial workflows, where generated tests must run reliably in downstream validation pipelines and unstable outputs increase inspection and repair costs. At the subject level, EvoLLM is not always the lowest-failure technique; on Fastjson, for example, it has a higher execution-failure rate than EvoSuite and CLAST. This suggests that cluster-wise refactoring still leaves room for robustness improvements on some subjects. Even so, the overall results indicate that EvoLLM substantially improves effectiveness while keeping execution-failure rates low.

> **Answer to RQ1.** EvoLLM is practically effective in modern Java settings. It remains deployable across JDK 11 and 17, also supports a large JDK 17-only Spring-based framework, achieves the strongest effectiveness among the compared techniques, and maintains lower average execution-failure rates than prior LLM-based techniques.

4.3 RQ2: LLM Inference Efficiency

Table 5 shows that EvoLLM is the most efficient technique among the compared approaches in terms of token usage. Across the nine subjects, it achieves the lowest total-token and total-tokens-per-test usage on every subject, and the lowest prompt-token and completion-token usage on eight of the nine subjects. On average, EvoLLM also uses the fewest tokens in all four categories. This suggests that EvoLLM reduces not only the amount of generated output, but also the amount of context sent to the model and the overhead of LLM interaction. In terms of total tokens, the advantage is consistent across all subjects; for example, on Fastjson, Spoon, and Common, EvoLLM uses 2.09M, 1.16M, and 1.14M tokens, respectively, whereas the closest baselines require 4.07M, 1.80M, and 2.44M or more.

Notably, this advantage holds even though EvoLLM provides compact PUT context to the LLM during refactoring, whereas prior LLM-based refactoring approaches such as UTGen and CLAST do not. This indicates that EvoLLM reduces token usage not by omitting behavior-relevant context, but by eliminating redundant prompts and reusing compact PUT context across clustered tests. In contrast, prior refactoring techniques operate at the individual-test level and often rely on multiple LLM calls per test to improve readability, for example, test-data rewriting, readability-oriented rewriting, and test naming. Such per-test, multi-stage prompting naturally accumulates both prompt and completion tokens.

> **Answer to RQ2.** EvoLLM achieves the lowest total and per-test token usage on every subject, the lowest average usage in all four token categories, and the lowest prompt and completion usage on eight of the nine subjects.

4.4 RQ3: Ablation and Controlled Comparison

To assess the effect of engineering hardening, we compare EvoSuite$_{ng}$ with the EvoSuite averages in Table 4. Under the same evaluation protocol, EvoSuite$_{ng}$ achieves average line coverage of

Table 5: Subject-wise LLM token usage.

Metric	Subject	ChatUT	HITS	CoverUp	UTGen	CLAST	EvoLLM
Prompt tokens	Fastjson	6.84M	13.35M	13.22M	2.08M	4.54M	**1.36M**
	Guava	2.23M	6.84M	6.98M	1.71M	2.96M	**1.03M**
	Seata	2.50M	8.98M	6.76M	0.58M	1.26M	**0.38M**
	Spoon	6.90M	12.16M	9.39M	1.48M	1.29M	**0.76M**
	Okio	1.61M	4.02M	2.78M	1.09M	3.63M	**0.96M**
	Weka	12.88M	22.63M	20.68M	2.95M	7.42M	**2.25M**
	Client	3.32M	12.87M	8.61M	1.10M	1.62M	**0.60M**
	Server	4.09M	20.41M	9.74M	**0.59M**	1.12M	0.61M
	Common	3.54M	17.78M	9.35M	1.45M	2.27M	**0.81M**
	Average	4.88M	13.23M	9.72M	1.45M	2.90M	**0.97M**
Compl. tokens	Fastjson	3.93M	7.47M	7.24M	2.00M	1.81M	**0.73M**
	Guava	1.61M	4.72M	4.97M	1.40M	1.18M	**0.61M**
	Seata	1.69M	5.79M	4.79M	0.45M	0.49M	**0.25M**
	Spoon	3.70M	6.29M	5.33M	1.41M	0.52M	**0.39M**
	Okio	1.29M	2.95M	2.20M	0.94M	1.52M	**0.40M**
	Weka	6.84M	12.45M	12.22M	3.06M	2.95M	**0.80M**
	Client	2.21M	8.17M	6.04M	0.89M	0.64M	**0.29M**
	Server	2.59M	13.14M	6.52M	0.58M	**0.43M**	0.46M
	Common	1.18M	5.06M	4.91M	0.99M	0.87M	**0.34M**
	Average	2.78M	7.34M	6.02M	1.30M	1.16M	**0.47M**
Total tokens	Fastjson	10.77M	20.82M	20.45M	4.07M	6.34M	**2.09M**
	Guava	3.84M	11.56M	11.95M	3.12M	4.15M	**1.64M**
	Seata	4.20M	14.77M	11.55M	1.03M	1.75M	**0.63M**
	Spoon	10.60M	18.44M	14.72M	2.89M	1.80M	**1.16M**
	Okio	2.89M	6.98M	4.98M	2.03M	5.15M	**1.36M**
	Weka	19.72M	35.09M	32.90M	6.01M	10.38M	**3.06M**
	Client	5.53M	21.04M	14.65M	1.99M	2.26M	**0.89M**
	Server	6.68M	33.55M	16.27M	1.17M	1.55M	**1.07M**
	Common	4.72M	22.83M	14.25M	2.44M	3.13M	**1.14M**
	Average	7.66M	20.56M	15.75M	2.75M	4.06M	**1.45M**
Total tokens / test	Fastjson	3,388	2,906	11,947	4,019	1,722	**629**
	Guava	1,148	979	4,340	3,101	1,601	**670**
	Seata	2,369	3,773	9,173	3,227	1,561	**792**
	Spoon	4,626	5,468	10,930	4,898	1,817	**697**
	Okio	1,074	916	2,808	3,649	1,740	**677**
	Weka	3,461	3,188	10,277	3,956	1,888	**928**
	Client	3,131	8,035	14,846	3,097	1,731	**707**
	Server	6,707	14,468	27,025	2,858	1,609	**925**
	Common	1,810	3,266	6,781	2,285	1,606	**684**
	Average	3,079	4,778	10,903	3,454	1,697	**745**

72.5%, branch coverage of 56.9%, mutation score of 50.6%, and an execution-failure rate of 0.28%, improving over EvoSuite by 9.2 pp (= 72.5 - 63.3), 9.9 pp (= 56.9 - 47.0), and 7.8 pp (= 50.6 - 42.8), while reducing execution-failure rate by 0.96 pp (= 1.24 - 0.28). These results show that engineering hardening is a major contributor to EvoLLM's effectiveness. In addition, applying the refactoring stage leaves these gains largely intact: EvoLLM remains within 1.0 pp of EvoSuite$_{ng}$ on line coverage (71.9 vs. 72.5), branch coverage (57.1 vs. 56.9), and mutation score (49.6 vs. 50.6).

To further isolate the contribution of the refactoring stage, we compare refactoring-based methods under a common EvoSuite baseline because end-to-end comparisons would otherwise mix refactoring effects with generator differences. UTGen is built on EvoSuite, CLAST refines EvoSuite-generated tests, whereas our refactoring stage is built on EvoSuite$_{ng}$. We therefore instantiate our refactoring stage on EvoSuite and compare it with UTGen and CLAST. Table 6 reports pp differences from EvoSuite, where values closer to zero indicate better preservation of baseline effectiveness. For the final output, which includes fallback to the original EvoSuite test when validation fails, our method shows the smallest drops: 1.42 pp in line coverage, 1.13 pp in branch coverage, and 2.58 pp in mutation score, compared with 5.80 pp, 4.52 pp, and 4.76 pp for UTGen, and 9.65 pp, 8.23 pp, and 17.71 pp for CLAST. Thus, under the same EvoSuite-based setting, our refactoring stage better preserves baseline effectiveness than the prior approaches.

Table 6: Controlled comparison of refactoring pipelines under a common EvoSuite baseline. Entries are percentage-point differences from EvoSuite (closer to zero is better). *Final output* includes tests that fall back to the original EvoSuite test; *Refactored only* excludes them.

Subset	Metric	UTGen	CLAST	Ours (on EvoSuite)
Final output (with fallback)	Δ Line coverage (pp)	-5.80	-9.65	**-1.42**
	Δ Branch coverage (pp)	-4.52	-8.23	**-1.13**
	Δ Mutation score (pp)	-4.76	-17.71	**-2.58**
Refactored only (fallback excluded)	Δ Line coverage (pp)	-20.74	-9.65	**-2.02**
	Δ Branch coverage (pp)	-19.46	-8.23	**-1.50**
	Δ Mutation score (pp)	-15.71	-17.71	**-3.13**

Table 7: Survey questions.

	Survey question
SQ1.	Is the purpose of the test easily understandable?
SQ2.	Do you comprehend the testing goal that the test is intended to achieve?
SQ3.	Does the test provide value for validating the program's behavior?
SQ4.	Does the name of the test accurately describe its testing goal?
SQ5.	Are the variable names within the test semantically meaningful?
SQ6.	Would you consider the test suitable for the unit-test suite?
SQ7.	Please provide any additional comments or feedback regarding the test.

EvoLLM and UTGen emit the original EvoSuite test when refactoring fails (e.g., due to LLM-response parsing failures, compilation errors, or execution failures). Because of this fallback, the final outputs may partly reflect the fallback behavior rather than the quality of the refactored tests alone. Thus, we exclude unrefactored tests and reevaluate only the refactored tests. Under this analysis (i.e., Refactored only in Table 6), our refactoring stage remains close to the original EvoSuite results, with drops of only 2.02 pp in line coverage, 1.50 pp in branch coverage, and 3.13 pp in mutation score. In contrast, UTGen incurs substantially larger drops of 20.74 pp, 19.46 pp, and 15.71 pp, while CLAST remains unchanged because it has no validation-and-revert step and emits every refactored test directly.

> **Answer to RQ3.** Engineering hardening substantially strengthens the baseline, and the proposed refactoring pipeline preserves most of that effectiveness while incurring much smaller losses than prior approaches.

4.5 RQ4: Readability

Study Design. To understand how practitioners perceive the readability of tests produced by EvoLLM relative to tests produced by prior approaches and developer-written tests, we conducted an industrial survey adapted from prior human-evaluation studies of code readability and generated-test understandability [13, 41, 51, 52, 58]. To reduce participant burden, we randomly selected eight

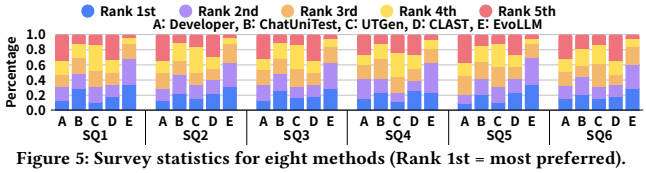


Figure 5: Survey statistics for eight methods (Rank 1st = most preferred).

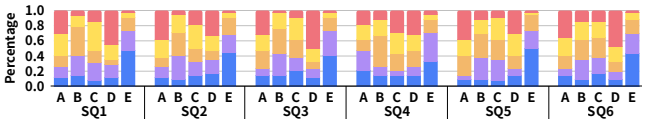


Figure 6: Survey statistics for three complex methods (65–100 LOC).

target methods (10–100 LOC) from our evaluation subjects. We capped the range at 100 LOC to keep the five-test ranking task manageable in a single sitting. For each method, participants were shown five tests: one generated by each tool (i.e., ChatUniTest, UTGen, CLAST, and EvoLLM) and one written by a developer. Our survey provided the target source code of each method with a brief description, so that participants could assess test readability with access to the program context. Participants then answered the survey questions in Table 7 by ranking the five tests for each question. Participants were blinded to the source technique of each test.

Results. We collected responses from 14 professional developers; 79% had over 10 years of software-development experience, and 57% had at least 10 years of industry experience. Figure 5 shows the ranking distributions across all survey questions. Across the questions, participants generally preferred EvoLLM-generated tests. For SQ1, 69% of rankings placed EvoLLM first or second, compared with ChatUniTest (49%), CLAST (34%), developer-written tests (31%), and UTGen (31%). One exception was SQ4, where CLAST received a slightly higher share of first-place rankings than EvoLLM.

SQ7 comments suggest that EvoLLM's readability advantage was less apparent for simple methods but became clearer for complex ones. Figure 6 focuses on three complex methods ranging from 65 to 100 LOC. For these methods, EvoLLM received the largest share of first-place rankings for every survey question. Participants' comments suggest that concise yet informative identifiers and well-placed comments were particularly helpful for understanding complex tests. Several comments also noted that excessively long test-method or variable names could reduce readability, particularly in CLAST-generated tests.

Interestingly, developer-written tests received the least favorable ranking distribution overall (Figure 5). One possible explanation from participant responses is that the original test authors, being familiar with the target methods, sometimes used less descriptive names or fewer explanatory comments. These observations suggest that EvoLLM-generated tests may be particularly helpful for readers who are less familiar with the target code, such as newly onboarded developers.

> **Answer to RQ4.** EvoLLM received the most favorable readability rankings among the compared test sources. Its readability advantages were especially pronounced for the more complex methods included in the study.

5 Lessons Learned

Even a Mature Test Generator Can Benefit Substantially from Engineering Hardening. Our experience suggests that research maturity and widespread use as a benchmark baseline do not necessarily translate into industrial readiness. Although EvoSuite has been extensively studied and widely used in Java unit-test generation research [14, 18–20, 44], applying it to modern Java projects still exposed coverage-limiting failures and deployment fragility. Our failure-analysis-driven hardening process addressed these issues and produced measurable gains in effectiveness and stability.

Generated Tests Must Be Readable to Serve as Maintainable Artifacts. In our preliminary use, developers often did not treat EvoSuite-generated tests as artifacts to be inspected and evolved. When a generated test was difficult to read, they tended to discard it and write a new test manually rather than spend effort inferring its intent, debugging failures, or modifying it. These observations suggest that automated unit-test generation must produce tests whose intent and exercised behavior are clear enough for developers to understand; otherwise, the generated tests may be ignored, substantially reducing the usefulness of the generation tool itself.

Token Efficiency Should Be Explicitly Considered in Industrial LLM-Based Test Generation. As enterprises increasingly adopt LLM-based development tools [3], token usage and GPU availability have become practical constraints for repeated industrial use [7, 24]. However, many prior LLM-based testing techniques primarily focus on coverage, repair, or readability, while treating token efficiency as a secondary concern [4, 13, 56, 58, 62]. In industrial settings, this cost matters because test-generation workflows must be applied repeatedly across many projects under shared GPU and operational budgets. EvoLLM addresses this constraint directly by making token efficiency an explicit design goal without dropping behavior-relevant program context. Rather than omitting PUT context, EvoLLM clusters related tests and shares compact PUT context within each cluster, reducing redundant LLM input while preserving the information needed for refactoring.

Treating the LLM as an Interchangeable Component Eases Deployment as Models Evolve. New and more capable LLMs are released rapidly [16]. A testing tool should therefore be able to adopt stronger models without being rebuilt. Fine-tuning may improve quality, but it makes each model upgrade require costly retraining and redeployment. This issue arose even during the short development and deployment of EvoLLM, as we migrated from Llama 3.1 [25] to Llama 3.3 [34] and then to gpt-oss [38]. We instead interact with the LLM only through prompting and external validation, allowing models to be replaced without retraining or changes to the core testing pipeline. Although EvoLLM is not specialized for a particular model, we consider the ability to benefit from continued model improvements more valuable for long-lived deployments.

6 Related Work

Search-Based Test Generation. Automated unit-test generation has long been studied as a way to reduce the manual effort of building regression test suites. SBST formulates test generation

as a search problem over coverage-oriented objectives and has been extensively studied for Java unit-test generation [2, 6, 8, 19–21, 31, 43, 44, 48]. EvoSuite [19] is an established SBST tool for Java that automatically generates JUnit tests and has been evaluated in both benchmark and industrial settings [2, 20]. However, SBST-generated tests can be difficult to read and maintain, which limits their usefulness as developer-facing artifacts in industrial settings. EvoLLM builds on the coverage strengths of SBST while using LLM-based refactoring to improve readability.

LLM-Based Test Generation. Recent work has explored LLMs for unit-test generation, leveraging their code understanding and synthesis capabilities [1, 4, 9, 10, 30, 41, 49, 55–57]. These techniques typically frame test generation as code synthesis conditioned on program context, prompts, examples, or feedback. AthenaTest [55] and A3Test [1] train neural models for unit-test generation, with A3Test incorporating assertion knowledge and test-signature verification to improve correctness. CodaMosa [30] combines SBST with LLM-based code synthesis, using LLM-generated snippets to help escape coverage plateaus. ChatUniTest [9] iteratively prompts an LLM to generate and repair tests based on execution feedback, while CoverUp [4] uses coverage feedback to prompt the LLM with uncovered code regions. Program-analysis-guided prompting has also been explored: HITS [56] uses method slicing, SymPrompt [49] uses code-aware prompting, TELPA [57] targets hard-to-cover branches using usage scenarios, dependency analysis, and counter-examples, and Aster [41] uses static analysis to guide generation toward coverage and readability. These techniques primarily use LLMs as test generators. In contrast, EvoLLM uses the LLM as a readability-oriented refactoring component on top of SBST-generated tests, preserving the coverage-oriented role of the underlying generator.

LLM-Based Test Refactoring. A closer line of work uses LLMs to improve existing tests rather than generate tests from scratch. Prior work has studied LLMs for test-smell detection [32], while LLM-based refactoring approaches aim to remove smells, restructure tests, or enhance readability [13, 23, 58]. UTGen [13] uses LLMs to improve the readability and semantic clarity of automatically generated unit tests by rewriting opaque test names, variable names, and structures. CLAST [58] combines program analysis with LLM-based refinement to decompose and restructure test cases into semantically clearer units. EvoLLM shares the goal of improving SBST-generated tests with LLM assistance, but differs in how it constructs refactoring inputs. Compared with these refactoring approaches, EvoLLM uses coverage goals to select goal-focused tests, form method-aware clusters, and share compact PUT context across related tests, thereby preserving SBST coverage while reducing redundant LLM input.

7 Threats to Validity

A potential threat concerns comparability under our evaluation setup. All techniques were evaluated under a class-level evaluation setting with a common model and budget, and this shared setup may not match each technique's best native configuration. For instance, fixing a single model (gpt-oss-20b) runs each baseline on a model different from the one used in its original evaluation. In addition, because public implementations released by the original authors of HITS and CoverUp were unavailable, we used the implementations provided through the ChatUniTest framework [9, 62]. To reduce this threat, we ran all techniques on JUGE, an established benchmarking infrastructure for Java unit-test generation used across multiple competition editions [15, 22, 27, 29, 45], under the same end-to-end protocol, and repeated each class-level experiment three times. We also made small prompt-level adjustments to match our execution environment—for instance, specifying the JUnit and Mockito versions on our classpath so that gpt-oss-20b generates compatible tests—without modifying the techniques themselves. Although these steps do not eliminate the possibility that some techniques would perform differently in their original or author-tuned settings, the main trends were consistent across target classes, subjects, and baseline categories, rather than being driven by a small number of cases.

Another threat concerns external validity. The three industrial subjects come from one organization and a related application ecosystem, and their proprietary source code cannot be released, limiting independent reproduction of the industrial results. The readability study also involved 14 developers and eight target methods. Although the six open-source subjects span multiple application domains and the readability study used experienced practitioners and blind comparisons, the industrial and readability findings may not fully generalize to other settings.

8 Conclusion

In this paper, we presented EvoLLM, a hybrid Java unit-test generation technique designed for industrial use. EvoLLM combines an engineering-hardened EvoSuite with cluster-wise LLM refactoring to retain SBST effectiveness, improve readability, support deployment on modern Java projects, and keep LLM token use low and predictable. Across six open-source and three industrial subjects, EvoLLM improved average line coverage by 8.2–20.7 pp, branch coverage by 6.2–19.4 pp, and mutation score by 6.8–24.5 pp over the compared techniques, while reducing token usage by 47.3%–93.0%. Its test-generation stage also demonstrated deployability across projects using JDKs 8, 11, and 17. A 14-participant developer study further suggested that EvoLLM-generated tests were perceived as more readable than tests produced by prior tools and developer-written tests.

For future work, we plan to extend EvoLLM to Java releases beyond JDK 17 and integrate it into our merge pipeline. We will also investigate test-regeneration policies and flaky-test handling in continuous integration, as well as the long-term maintainability of generated tests.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback. This work was supported by Samsung SDS, the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.2020-0-01337,(SW STAR LAB) Research on Highly-Practical Automated Software Repair(10%), IITP-2026-RS-2020-II201819(5%)), and the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(No.2021R1A5A1021944(10%)).

Data Availability Statement

The experimental data used in this paper are archived at: https://doi.org/10.5281/zenodo.19891585. Proprietary industrial source code and internal project identifiers are excluded for confidentiality.

References

[1] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test case generation. *Information and Software Technology* 176 (2024), 107565. doi:10.1016/j.infsof.2024.107565

[2] M Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 263–272. doi:10.1109/ICSE-SEIP.2017.27

[3] Altman Solon. 2024. Enterprise Generative AI in 2024: The Future of Work. https://www.altmansolon.com/thought-leadership/2024-enterprise-adoption-generative-ai. Accessed: 2026-04-24.

[4] Juan Altmayer Pizzorno and Emery D Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. doi:10.1145/3729398

[5] Anthropic. [n. d.]. Claude API Pricing. https://platform.claude.com/docs/en/about-claude/pricing. Accessed: 2026-04-28.

[6] Andrea Arcuri. 2017. Many independent objective (MIO) algorithm for test suite generation. In *Search Based Software Engineering: 9th International Symposium, SSBSE 2017, Paderborn, Germany, September 9-11, 2017, Proceedings 9*. Springer, 3–17. doi:10.1007/978-3-319-66299-2_1

[7] Alistair Barr. 2025. AI Is Changing How Software Companies Charge Customers. Welcome to the Pay-as-You-Go Future. https://www.businessinsider.com/saas-ai-changing-how-software-companies-charge-customers-2025-4. Business Insider. Accessed: 2026-04-24.

[8] José Campos, Andrea Arcuri, Gordon Fraser, and Rui Abreu. 2014. Continuous Test Generation: Enhancing Continuous Integration with Automated Test Generation. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering* (Vasteras, Sweden) *(ASE '14)*. Association for Computing Machinery, New York, NY, USA, 55–66. doi:10.1145/2642937.2643002

[9] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A framework for LLM-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576. doi:10.1145/3663529.3663801

[10] Xiang Cheng, Fan Sang, Yizhuo Zhai, Xiaokuan Zhang, and Taesoo Kim. 2025. Rug: Turbo Llm for Rust Unit Test Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 2983–2995. doi:10.1109/ICSE55347.2025.00097

[11] Ermira Daka, José Campos, Gordon Fraser, Jonathan Dorn, and Westley Weimer. 2015. Modeling readability to improve unit tests. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 107–118. doi:10.1145/2786805.2786838

[12] Ermira Daka and Gordon Fraser. 2014. A Survey on Unit Testing Practices and Problems. In *Proceedings of the 2014 IEEE 25th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 201–211. doi:10.1109/ISSRE.2014.11

[13] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2025. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)*. 1449–1461. doi:10.1109/ICSE55347.2025.00032

[14] Xavier Devroey, Alessio Gambi, Juan Pablo Galeotti, René Just, Fitsum Kifetew, Annibale Panichella, and Sebastiano Panichella. 2023. JUGE: An Infrastructure for Benchmarking Java Unit Test Generators. *Software Testing, Verification and Reliability* 33, 3 (2023), e1838. doi:10.1002/stvr.1838

[15] Xavier Devroey, Sebastiano Panichella, and Alessio Gambi. 2020. Java Unit Testing Tool Competition—Eighth Round. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops (ICSEW '20)*. ACM, 545–548. doi:10.1145/3387940.3392265

[16] Epoch AI. 2025. AI Capabilities Have Steadily Improved over the Past Year. https://epoch.ai/data-insights/ai-capabilities-over-past-year. Accessed: 2026-07-27.

[17] Martin Fowler. 2013. GivenWhenThen. https://martinfowler.com/bliki/GivenWhenThen.html. Accessed: 2026-05-01.

[18] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419. doi:10.1145/2025113.2025179

[19] Gordon Fraser and Andrea Arcuri. 2013. Whole Test Suite Generation. *IEEE Transactions on Software Engineering* 39, 2 (2013), 276–291. doi:10.1109/TSE.2012.14

[20] Gordon Fraser and Andrea Arcuri. 2014. A large-scale evaluation of automated unit test generation using evosuite. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 2 (2014), 1–42. doi:10.1145/2685612

[21] Gordon Fraser, Matt Staats, Phil McMinn, Andrea Arcuri, and Frank Padberg. 2013. Does Automated White-Box Test Generation Really Help Software Testers?. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis* (Lugano, Switzerland) *(ISSTA 2013)*. Association for Computing Machinery, New York, NY, USA, 291–301. doi:10.1145/2483760.2483774

[22] Alessio Gambi, Gunel Jahangirova, Vincenzo Riccio, and Fiorella Zampetti. 2022. SBST Tool Competition 2022. In *Proceedings of the 15th Workshop on Search-Based Software Testing (SBST '22)*. ACM, 25–32. doi:10.1145/3526072.3527538

[23] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2025. Automated unit test refactoring. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 713–733. doi:10.1145/3715750

[24] Sharon Goldman. 2026. Even Nvidia's Own Research Teams Can't Get Enough GPUs Amid the Race for AI Computing Power. https://fortune.com/2026/04/09/nvidia-gpu-shortage-impacts-even-nvidias-own-research-teams-bryan-catanzaro-eye-on-ai/. Fortune. Accessed: 2026-04-24.

[25] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). doi:10.48550/arXiv.2407.21783

[26] Sergey Ioffe. 2010. Improved Consistent Sampling, Weighted Minhash and L1 Sketching. In *2010 IEEE International Conference on Data Mining*. 246–255. doi:10.1109/ICDM.2010.80

[27] Gunel Jahangirova and Valerio Terragni. 2023. SBFT Tool Competition 2023 - Java Test Case Generation Track. In *2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)*. 61–64. doi:10.1109/SBFT59156.2023.00025

[28] Karen Sparck Jones. 1972. A Statistical Interpretation of Term Specificity and Its Application in Retrieval. *Journal of Documentation* 28, 1 (1972), 11–21. doi:10.1108/eb026526

[29] Fitsum Kifetew, Lin Yun, and Davide Prandi. 2025. SBFT Tool Competition 2025 – Java Test Case Generation Track. arXiv:2504.09168 [cs.SE] https://arxiv.org/abs/2504.09168

[30] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931. doi:10.1109/ICSE48619.2023.00085

[31] Yun Lin, You Sheng Ong, Jun Sun, Gordon Fraser, and Jin Song Dong. 2021. Graph-Based Seed Object Synthesis for Search-Based Unit Testing *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1068–1080. doi:10.1145/3468264.3468619

[32] Keila Lucas, Rohit Gheyi, Elvys Soares, Márcio Ribeiro, and Ivan Machado. 2024. Evaluating Large Language Models in Detecting Test Smells. arXiv:2407.19261 [cs.SE] https://arxiv.org/abs/2407.19261

[33] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. doi:10.1017/CBO9780511809071

[34] Meta AI. 2024. Llama 3.3 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md. Accessed: 2026-07-27.

[35] Seokhyeon Moon and Yoonchan Jhi. 2024. EvoFuzz at the SBFT 2024 Tool Competition. In *Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing (SBFT '24)*. ACM, 63–64. doi:10.1145/3643659.3648556

[36] Zifan Nan, Zhaoqiang Guo, Kui Liu, and Xin Xia. 2025. Test intention guided llm-based unit test generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1026–1038. doi:10.1109/ICSE55347.2025.00243

[37] OpenAI. [n. d.]. OpenAI API Pricing. https://developers.openai.com/api/docs/pricing. Accessed: 2026-04-28.

[38] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv preprint arXiv:2508.10925* (2025). doi:10.48550/arXiv.2508.10925

[39] Oracle. 2021. Java Language Changes for Java SE 17. https://docs.oracle.com/en/java/javase/17/language/java-language-changes.html. Accessed: 2026-04-29.

[40] Oracle. 2021. The Java Virtual Machine Specification, Java SE 17 Edition, Chapter 4: The class File Format. https://docs.oracle.com/javase/specs/jvms/se17/html/jvms-4.html. Accessed: 2026-04-29.

[41] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2025. Aster: Natural and multi-language unit test generation with llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 413–424. doi:10.1109/ICSE-SEIP66354.2025.00042

[42] Annibale Panichella, José Campos, and Gordon Fraser. 2020. Evosuite at the SBST 2020 tool competition. In *Proceedings of the IEEE/ACM 42nd international conference on software engineering workshops*. 549–552. doi:10.1145/3387940.3392266

[43] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2015. Reformulating Branch Coverage as a Many-Objective Optimization Problem. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. 1–10. doi:10.1109/ICST.2015.7102604

[44] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2017. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering* 44, 2 (2017), 122–158. doi:10.1109/TSE.2017.2663435

[45] Sebastiano Panichella, Alessio Gambi, Fiorella Zampetti, and Vincenzo Riccio. 2021. SBST tool competition 2021. In *2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST)*. IEEE, 20–27. doi:10.1109/SBST52555.2021.00011

[46] Mark Reinhold. 2017. JEP 261: Module System. https://openjdk.org/jeps/261. Accessed: 2026-04-29.

[47] Mark Reinhold. 2021. JEP 403: Strongly Encapsulate JDK Internals. https://openjdk.org/jeps/403. Accessed: 2026-04-29.

[48] José Miguel Rojas, Gordon Fraser, and Andrea Arcuri. 2016. Seeding strategies in search-based unit test generation. *Software Testing, Verification and Reliability* 26, 5 (2016), 366–401. doi:10.1002/stvr.1601

[49] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971. doi:10.1145/3643769

[50] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105. doi:10.1109/TSE.2023.3334955

[51] Agnia Sergeyuk, Olga Lvova, Sergey Titov, Anastasiia Serova, Farid Bagirov, and Timofey Bryksin. 2024. Assessing Consensus of Developers' Views on Code Readability. In *Proceedings of the 35th Annual Workshop of the Psychology of Programming Interest Group*. Psychology of Programming Interest Group, 37–44. https://ppig.org/files/2024-PPIG-35th-sergeyuk.pdf

[52] Agnia Sergeyuk, Olga Lvova, Sergey Titov, Anastasiia Serova, Farid Bagirov, Evgeniia Kirillova, and Timofey Bryksin. 2024. Reassessing Java code readability models with a human-centered approach. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 225–235. doi:10.1145/3643916.3644435

[53] Sina Shamshiri, José Miguel Rojas, Juan Pablo Galeotti, Neil Walkinshaw, and Gordon Fraser. 2018. How Do Automatically Generated Unit Tests Influence Software Maintenance?. In *Proceedings of the 11th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 250–261. doi:10.1109/ICST.2018.00033

[54] Spring Framework. 2026. Spring Framework Overview. https://docs.spring.io/spring-framework/reference/overview.html. Accessed: 2026-04-29.

[55] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2021. Unit Test Case Generation with Transformers and Focal Context. arXiv:2009.05617 [cs.SE] https://arxiv.org/abs/2009.05617

[56] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268. doi:10.1145/3691620.3695501

[57] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. arXiv:2404.04966 [cs.SE] https://arxiv.org/abs/2404.04966

[58] Chen Yang, Lin Yang, Ziqi Wang, Dong Wang, Jianyi Zhou, and Junjie Chen. 2025. Clarifying Semantics of In-Context Examples for Unit Test Generation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3046–3057. doi:10.1109/ASE63991.2025.00250

[59] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1607–1619. doi:10.1145/3691620.3695529

[60] Shin Yoo and Mark Harman. 2012. Regression Testing Minimization, Selection and Prioritization: A Survey. *Software Testing, Verification and Reliability* 22, 2 (2012), 67–120. doi:10.1002/stv.430

[61] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving ChatGPT for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726. doi:10.1145/3660783

[62] ZJU-ACES-ISE. 2023. ChatUniTest: An Innovative Framework for LLM-Based Unit Test Generation. https://github.com/ZJU-ACES-ISE/ChatUniTest. Accessed: 2026-04-22.

Received 2026-05-01; accepted 2026-07-01